

Lab0 实验报告

张雪阳 25301030086

作业一：

问题一：

我之前没有正式的多人协同开发经历。在学习 Git 之后，了解到多人协作一般这样分工：

- ①每个开发者基于主分支（main/master）新建自己的功能分支，在独立分支上开发新功能；
- ②开发完成后提交代码，发起合并请求（Pull Request）；
- ③其他成员进行代码审查，确认无误后再合并到主分支；
- ④所有人定期拉取远程仓库最新代码，保持本地代码同步，减少代码冲突。

问题二：

Git 把操作拆成暂存和提交两步，主要目的：

- ①选择性提交：可以只把部分修改的文件加入暂存区，一次 commit 只提交一部分改动，不需要把当前所有文件变更一次性提交，方便把修改拆成多个逻辑独立的提交记录。
- ②预览检查：加入暂存区后，可以查看暂存区里的修改内容，确认无误后再 commit，避免误提交不需要的文件。
- ③灵活管理变更：暂存区是一个中间缓冲区，可以反复添加、撤销文件，自由组织本次要提交的内容，让每一次 commit 对应一个完整、独立的功能改动，保证提交历史清晰可读。

问题三：

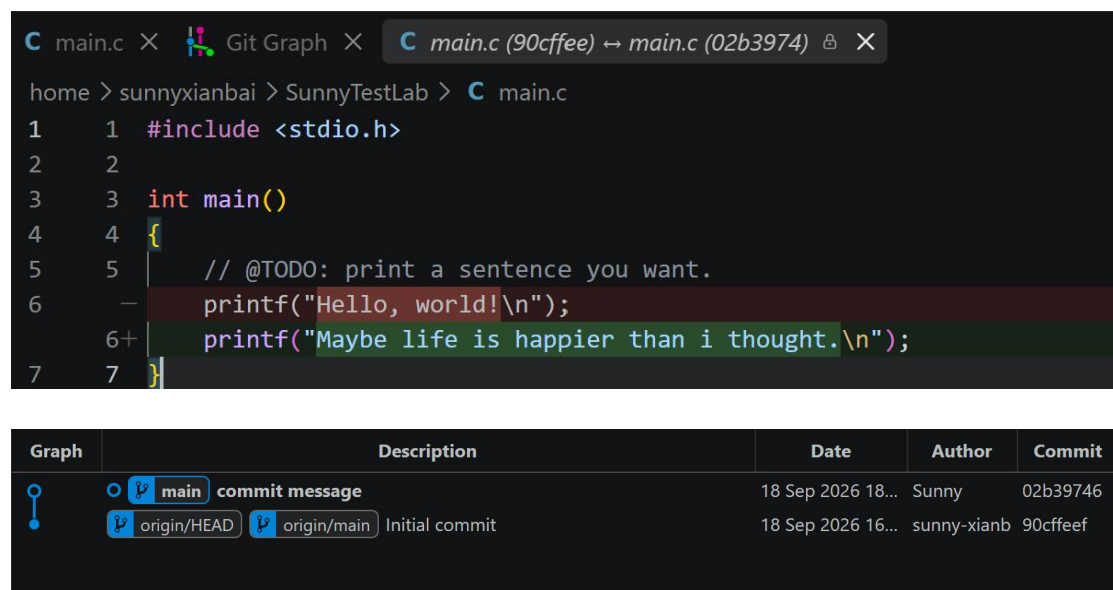
git branch：只列出本地分支，看不到远程仓库的分支。

git branch -a：列出所有分支：本地分支 + 远程仓库上的分支。

执行 git branch，只能看到电脑上创建的分支。

执行 git branch -a，除本地分支外，还会显示远程分支，格式类似 remotes/origin/main，代表 GitHub 远程仓库上的分支。

作业二：



The screenshot displays a code editor with a C program and a Git commit history table.

Code Editor:

```
main.c X Git Graph X C main.c (90cffee) ↔ main.c (02b3974) X
home > sunnyxianbai > SunnyTestLab > C main.c
1 1 #include <stdio.h>
2 2
3 3 int main()
4 4 {
5 5     // @TODO: print a sentence you want.
6 - printf("Hello, world!\n");
6+ printf("Maybe life is happier than i thought.\n");
7 7 }
```

Git Commit History Table:

Graph	Description	Date	Author	Commit
	 main commit message	18 Sep 2026 18...	Sunny	02b39746
	 origin/HEAD  origin/main Initial commit	18 Sep 2026 16...	sunny-xianb	90cffee

作业三:

1. Commit Message 规范

这篇文章介绍 Git 提交说明 (Commit Message) 的编写规范。完整提交信息分为 Header (头部, 必填)、Body (主体)、Footer (脚注)。Header 格式: type(scope): subject
type: feat 新增功能、fix 修复 bug、docs 文档、style 格式调整、refactor 重构、test 测试、chore 构建相关; scope: 可选, 说明改动影响范围; subject: 简短描述改动, 使用祈使句, 不写句号。

规范的提交信息好处: 方便查阅版本历史、快速定位代码变更; 可以自动生成 ChangeLog 更新日志, 在发布新版本时自动汇总新增功能、bug 修复、破坏性改动。如果提交信息随意填写, 版本历史难以阅读, 无法自动生成日志。

2. Gitflow 使用规范

Gitflow 是分支管理工作流, 定义 5 类分支: 永久分支 master 存放线上稳定代码、develop 作为开发集成主线; 临时分支 feature 开发新功能、release 准备版本发布、hotfix 修复线上紧急 bug。feature 从 develop 分出, 完成合并回 develop; release 由 develop 创建, 测试完成同时合并到 master 与 develop; hotfix 从 master 分出, 修复后同步合并 master、develop。上线时在 master 打 tag 记录版本。该流程隔离开发、测试、线上修复, 减少团队协作冲突, 保障生产代码稳定。

3. 为什么要学习 Git :

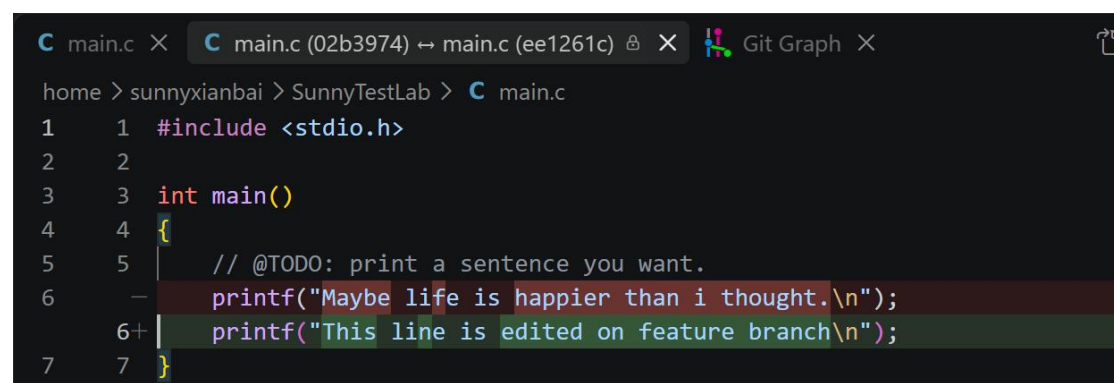
Git 是分布式版本控制系统。第一, Git 完整保存每一次提交快照, 可以回溯历史版本, 防止代码丢失; 第二, Git 的分支能力支持多人并行开发, 不同功能在独立分支开发, 出现冲突可以手动解决, 实现团队协作; 第三, 搭配 Commit Message 规范、Gitflow 分支策略, 代码改动、版本流转全部留下可读记录, 方便追溯问题、自动生成更新日志; 第四, 可以推送至远程仓库实现备份, 防止本地文件损坏。掌握 Git 与配套开发规范, 能够让个人和团队的开发过程有序可控, 提升代码维护与协作的效率。

作业四:

①在 Git 仓库中, 基于 main 主分支新建 feature 分支。

```
sunnyxianbai@SunnyLapTop:~/SunnyTestLab$ git checkout -b feature
Switched to a new branch 'feature'
```

②切换到 feature 分支, 修改 main.c 内 printf 语句, 保存文件, 添加至暂存区并提交。提交备注: feat: modify printf on feature branch



```
C main.c X C main.c (02b3974) ↔ main.c (ee1261c) X Git Graph X
home > sunnyxianbai > SunnyTestLab > C main.c
1 1 #include <stdio.h>
2 2
3 3 int main()
4 4 {
5 5     // @TODO: print a sentence you want.
6 - printf("Maybe life is happier than i thought.\n");
6+ printf("This line is edited on feature branch\n");
7 7 }
```

Branches: Show All		☒ Show Remote Branches			
Graph	Description		Date	Author	Commit
	feature	feat: modify printf on feature branch	18 Sep 2026 19...	Sunny	ee1261c8
	main	commit message	18 Sep 2026 18...	Sunny	02b39746
	origin/HEAD origin/main	Initial commit	18 Sep 2026 16...	sunny-xianb	90cfeef

③切换回 main 分支，修改同一行 printf 代码，保存并提交。提交备注：feat: modify printf on main branch

```
sunnyxianbai@SunnyLapTop:~/SunnyTestLab$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
```

```
home > sunnyxianbai > SunnyTestLab > C main.c
1 1 #include <stdio.h>
2 2
3 3 int main()
4 4 {
5 5     // @TODO: print a sentence you want.
6 - printf("Hello, world!\n");
6+ printf("This change comes from main branch\n");
7+
7 8 }
```

④保持在 main 分支，执行合并命令，将 feature 分支合并进 main 分支：

```
sunnyxianbai@SunnyLapTop:~/SunnyTestLab$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

⑤查看冲突标记，手动解决冲突 打开 main.c，Git 自动插入冲突分隔标记：

```
C main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
6 <<<<<<< HEAD (当前更改)
7     printf("This change comes from main branch\n");
8
9     =====
10    printf("This line is edited on feature branch\n");
11    >>>>>>> feature (传入的更改)
12 }
```

人工选择需要保留的代码，删除全部冲突标记，保存文件。

```
C main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6
7      printf("This change comes from main branch\n");
8
9
10
11 }
```

⑥提交冲突解决结果 将修改后的 main.c 加入暂存区，完成本次提交，提交备注：commit message。 提交完成后刷新 Git Graph，两条分支线条汇合，合并完成。

Graph	Description	Date	Author	Commit
	 main commit message	18 Sep 2026 19...	Sunny	df64cdc4
	feat: modify printf on main branch	18 Sep 2026 19...	Sunny	f56ceb25
	 feature feat: modify printf on feature branch	18 Sep 2026 19...	Sunny	ee1261c8
	commit message	18 Sep 2026 18...	Sunny	02b39746
	 origin/HEAD  origin/main Initial commit	18 Sep 2026 16...	sunny-xianb	90cffeef